


**UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA**

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
**INFORMATICA**

# Sviluppo di uno strumento di migrazione dati Infor LN

**Relatore:**
Prof. Massazza Paolo

**Tesi di Laurea di:**
Gabriele Caretti
Matricola 756564

**Anno accademico:**
2025/2026

# Indice


**1 Introduzione** **1**

1.1 Contesto Aziendale . . . . . 1

1.2 Obiettivi del Progetto . . . . . 1

1.3 Struttura della Tesi . . . . . 1

**2 Stato Attuale** **3**

2.1 Sistemi ERP . . . . . 3

2.1.1 Evoluzione dai Sistemi Legacy al Cloud . . . . . 3

2.2 Infor LN e CloudSuite Enterprise . . . . . 3

2.2.1 Il Linguaggio Baan C: 3GL e 4GL . . . . . 4

2.2.2 Infor Exchange e Strumenti di Import/Export . . . . . 4

2.3 Migrazione Dati nei Sistemi ERP . . . . . 5

2.3.1 Sfide e Problematiche Comuni . . . . . 5

**3 Analisi dei Requisiti** **6**

3.1 Scenario di Riferimento . . . . . 6

3.1.1 Il Flusso di Migrazione Esistente . . . . . 6

3.2 Requisiti Funzionali . . . . . 6

3.2.1 Sviluppo dell'Importer Infor Exchange . . . . . 6

3.2.2 Sistema di Estendibilità con Funzioni Ad-hoc . . . . . 7

3.2.3 Integrazione con lo Strumento Documentale . . . . . 7

3.3 Requisiti Non Funzionali . . . . . 7

3.3.1 Manutenibilità ed Estensibilità . . . . . 7

3.3.2 Sicurezza e Data Integrity in Multitenancy . . . . . 7

3.4 Modello dei Dati (Entità della Migrazione) . . . . . 8

3.4.1 Analisi delle Strutture Dati Coinvolte . . . . . 8

**4 Architettura** **9**

4.1 Panoramica dell'Architettura di Sistema . . . . . 9

4.2 Progettazione del Software . . . . . 9

4.2.1 Design dell'Importer e del Parser dei Dati . . . . . 9

4.2.2 Motore di Generazione del Codice . . . . . 9
4.2.3 Modulo di Integrazione Ad-hoc e Callbacks . . . . . 10
4.3 Scelte Tecnologiche . . . . . 10
4.3.1 Linguaggi e Framework . . . . . 10
4.3.2 Librerie di Supporto e Strumenti di Build . . . . . 10

**5 Implementazione** **11**
5.1 Sviluppo dell'Importer Infor Exchange . . . . . 11
5.1.1 Gestione dei Formati e Parsing . . . . . 11
5.1.2 Ricostruzione delle Definizioni dei File da Migrare . . . . 11
5.2 Implementazione del Generatore di Codice . . . . . 12
5.2.1 Template Engine e Creazione Dinamica . . . . . 12
5.2.2 Supporto alle Funzioni di Trasformazione Ad-hoc . . . . 12
5.3 Interfacce Utente (Sessioni) . . . . . 12
5.4 Integrazione Documentale . . . . . 13
5.4.1 Esportazione delle Specifiche di Migrazione . . . . . 13

**6 Testing e Validazione** **14**
6.1 Strategia di Testing . . . . . 14
6.1.1 Test Unitari sui Moduli di Parsing . . . . . 14
6.1.2 Test di Integrazione del Flusso di Migrazione . . . . . 14
6.2 Verifica dell'Integrità dei Dati . . . . . 15
6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud) 15
6.3 Metriche e Miglioramenti Prestazionali . . . . . 15

**7 Conclusioni** **16**
7.1 Obiettivi Raggiunti e Risultati Ottenuti . . . . . 16
7.2 Sfide Incontrate . . . . . 16
7.3 Sviluppi Futuri . . . . . 17

**Bibliografia** **18**

# Capitolo 1

# Introduzione

## 1.1 Contesto Aziendale

Il progetto di tesi si propone di sviluppare uno strumento di migrazione dati per Infor LN [3], un sistema ERP (Enterprise Resource Planning) industriale utilizzato a livello internazionale. L'obiettivo è agevolare il processo di migrazione da sistemi legacy a Infor LN, garantendo l'integrità e la coerenza dei dati in ambienti multitenant [1, 2].

## 1.2 Obiettivi del Progetto

Il lavoro si concentrerà sull'analisi dei requisiti, la progettazione dell'architettura del software, la stesura della documentazione e lo sviluppo di un prototipo funzionante. In particolare, lo strumento sarà in grado di generare dinamicamente codice, estendibile con funzioni ad hoc, per migrare dati su sistemi Cloud Multitenant, con un focus sulla Infor Cloudsuite Enterprise. Inoltre, è richiesto lo sviluppo di un importer capace di interpretare il formato di import/export di Infor Exchange al fine di ricreare le definizioni dei file da migrare, integrandolo con lo strumento aziendale già esistente per la produzione di specifiche di migrazione documentate.

## 1.3 Struttura della Tesi

L'elaborato si articola nel seguente modo:

- **Capitolo 2**: stato attuale degli ERP moderni, architetture cloud multitenant e sfide di migrazione dati.

- **Capitolo 3**: analisi dello scenario di partenza e dei requisiti funzionali e non funzionali.
- **Capitolo 4**: architettura del sistema, design e scelte tecnologiche (come Importer e Generatore codice).
- **Capitolo 5**: descrizione dell'implementazione concreta del progetto.
- **Capitolo 6**: testing, inclusi elementi come la validazione dell'integrità dei dati.
- **Capitolo 7**: conclusioni, traguardi raggiunti e possibili sviluppi futuri.

# Capitolo 2

# Stato Attuale

## 2.1 Sistemi ERP

I sistemi ERP (Enterprise Resource Planning) rappresentano un elemento essenziale del sistema gestionale aziendale moderno [4], consentendo l'integrazione dei processi di business rilevanti dell'azienda (vendite, acquisti, gestione inventario, contabilità).

### 2.1.1 Evoluzione dai Sistemi Legacy al Cloud

Nel tempo, i sistemi ERP si sono evoluti partendo da architetture on-premise legacy, spesso caratterizzate da un elevato grado di personalizzazione (customization, in Infor LN spesso definiti come tx) e importanti costi di mantenimento. Tuttavia, oggi si sta assistendo ad uno spostamento verso nuove soluzioni Cloud.

Il modello Cloud Multitenant rappresenta l'architettura degli ERP odierni [1]. Ove, una singola istanza del software e infrastruttura servono più clienti (definiti come tenant). Questo approccio offre scalabilità, aggiornamenti continui e ottimizzazione dei costi, ma anche diverse sfide nell'ambito della gestione e segregazione dei dati, nonché della sicurezza, richiedendo che ogni processo di migrazione operi secondo specifici vincoli di integrità [5].

## 2.2 Infor LN e CloudSuite Enterprise

Infor LN, nato con il nome di Baan ERP, è un ERP leader nel settore manifatturiero [3] a livello internazionale. Viene integrato all'interno della Infor CloudSuite Enterprise, il quale offre funzionalità avanzate per la gestione della supply chain e produzione [2].

Infor LN è basato su un'architettura che separa il framework tecnologico (gli Infor Tools) dalla logica di business. Ogni applicazione è composta da svariati oggetti:

- **Tabelle (Tables):** I dati fisici, definiti tramite Data Dictionary.
- **Sessioni (Sessions):** Le interfacce utente (da qui definite come Sessioni) attraverso cui gli operatori accedono, visualizzano e interagiscono con i dati.
- **DAL (Data Access Layer):** L'elemento che si occupa di garantire l'integrità dei dati e delle tabelle, indipendentemente dalle Sessioni.

### 2.2.1 Il Linguaggio Baan C: 3GL e 4GL

Per lo sviluppo e personalizzazione, Infor LN utilizza un linguaggio proprietario chiamato **Baan C**. Esso si divide a sua volta in due versioni:

- **4GL (Fourth-Generation Language):** La versione più recente e moderna. Infor LN Tools genera automaticamente operazioni standard di Sessione e database (CRUD). In 4GL non è necessario scrivere codice per gestire il salvataggio o il caricamento di un record, ma ci si concentra sulla gestione degli *eventi* legati ai componenti della Sessione o del database (tramite DAL).
- **3GL (Third-Generation Language):** Precedente a 4GL, e totalmente integrato con esso, viene utilizzato per operazioni come gestione di logiche di business complesse. Permette un approccio più procedurale e a basso livello per implementare funzioni matematiche, algoritmi personalizzati, DLL (Dynamic Link Libraries) proprietarie, funzioni e routine di manipolazione stringhe o file.

### 2.2.2 Infor Exchange e Strumenti di Import/Export

Per lo scambio di informazioni, Infor fornisce un modulo denominato *Exchange*. Si tratta di una piattaforma per l'importazione e l'esportazione di file (es. CSV, XML, testo a larghezza fissa) mappando righe e colonne sui *Datafiles* e sui campi del database di LN. Spesso si necessitano delle personalizzazioni delle procedure di migrazione tramite Baan C, che quindi risulta essenziale

quando le regole di mappatura superano la complessità gestibile nativamente dall'Exchange.

## 2.3 Migrazione Dati nei Sistemi ERP

### 2.3.1 Sfide e Problematiche Comuni

La migrazione da un sistema legacy a un nuovo ERP cloud, richiede un'attenta fase di analisi per evitare perdite o disallineamenti di informazioni storiche e master data. Attualmente, molte migrazioni sono per la maggior parte gestite manualmente, con l'integrazioni di script scritti ad hoc, con tutti i rischi che ne conseguono. Per questo, nasce l'idea del Tool di Migrazione oggetto di questa tesi.

Verso il Cloud, sono necessari tool robusti in grado di lavorare sia con logiche batch sia che con API e servizi web, adattandosi alle regole di business e garantendo la qualità di ogni singolo dato prima ancora che venga scritto nel database del nuovo tenant.

# Capitolo 3

# Analisi dei Requisiti

## 3.1 Scenario di Riferimento

### 3.1.1 Il Flusso di Migrazione Esistente

Negli ambienti legacy, il processo di migrazione dei dati [5] era basato su procedure manuali e script ad hoc. L'operatore (noi) era costretto a sviluppare script personalizzati per ogni singola tabella, con lunghi tempi di realizzazione e rischi inerenti alla trascrizione manuale delle logiche di mapping.

Inoltre, cambiamenti da parte di Infor e nuovi standard più restrittivi hanno reso le operazioni manuali molto più complesse [4]. Perciò, è emersa la necessità aziendale di innovare, ottimizzare e automatizzare il processo con un nuovo strumento, ossia l'integrazione di un *Migration Tool* parametrico.

## 3.2 Requisiti Funzionali

I requisiti funzionali definiscono i comportamenti attesi, ossia che cosa dovrebbe fare, del nuovo strumento, così come stabiliti con gli stakeholder.

### 3.2.1 Sviluppo dell'Importer Infor Exchange

Il sistema dovrà supportare l'input di file e strutture generate dallo strumento preesistente nativo di Infor Exchange. Tali file presentano specifici layout (come formati testuali ASCII o delimitati) che dovranno essere interpretati per popolare la gerarchia del dizionario `txamg110` (Datafiles) e `txamg111` (DataFile Fields).

Il punto cruciale della soluzione sarà lo strumento generatore di codice. Partendo dalle tabelle di mapping (es. `txamg203`), lo strumento dovrà generare

script di elaborazione e conversione on-the-fly, senza intervento manuale di programmazione.

### 3.2.2 Sistema di Estendibilità con Funzioni Ad-hoc

Nonostante l'automazione, alcune regole di business legacy non potranno essere collegate a semplici assegnazioni. Viene quindi l'esigenza di una funzionalità di callback per permettere, ad opera dello sviluppatore, l'inserimento di script personalizzati che verranno associati al processo principale durante l'esecuzione batch (`txamg3200`).

### 3.2.3 Integrazione con lo Strumento Documentale

Per garantire l'allineamento con gli standard del progetto, il *Migration Tool* dovrà interfacciarsi con la documentale aziendale, permettendo la produzione ed esportazione di specifiche tecniche di migrazione, mappature, eccezioni e log generati in fase di esecuzione (utili per effettuare del debugging).

## 3.3 Requisiti Non Funzionali

I requisiti non funzionali gestiscono i vincoli qualitativi e infrastrutturali del software prodotto.

### 3.3.1 Manutenibilità ed Estensibilità

Il *Migration Tool* dovrà essere interamente sviluppato in linguaggio Baan C rispettando il framework Infor Tools, così da garantire una stretta coerenza alle future release dell'ERP e una più semplice manutenibilità per il comparto IT aziendale.

### 3.3.2 Sicurezza e Data Integrity in Multitenancy

Dato che il sistema tratterà dati di tipo core dell'azienda cliente in contesti Cloud, la sicurezza e la segregazione (isolamento) dei dati per i *Tenant* costituiscono priorità assolute e fondamentali.

Inoltre, il caricamento dovrà ottimizzare l'uso della memoria e delle sessioni, prediligendo una elaborazione a blocchi al fine di minimizzare i tempi morti

dati dalla latenza di rete durante la connessione verso i server CloudSuite remoti.

## 3.4 Modello dei Dati (Entità della Migrazione)

### 3.4.1 Analisi delle Strutture Dati Coinvolte

Per supportare in modo efficace il processo di migrazione, è stata definita un'architettura dati articolata, basata su un package personalizzato e specifico per questo scopo. Il modello dei dati è suddiviso logicamente in diverse entità principali:

- **Parametrizzazione generale:**: tramite tabelle di configurazione come `txamg000` (AlteaIN Migration Tool Parameters).
- **Gestione dei Datafile:** tabelle come `txamg110` (Datafiles) e `txamg111` (DataFile Fields) permettono di definire le strutture dei file originari da importare.
- **Elaborazione a Lotti (Batch):** la configurazione e l'esecuzione tramite batch è regolata da tabelle come `txamg200` (Batch), `txamg201` (Batch Sequences) e `txamg202` (Target Tables), insieme ad altre tabelle dedicate al mapping dei campi (`txamg203`).
- **Tracciamento delle Esecuzioni:** lo stato e l'esito delle operazioni, oltre all'eventuale presenza di errori, vengono memorizzati all'interno delle tabelle `txamg300`, `txamg301`, `txamg302` e `txamg303` (Runs Errors).

I domini applicativi associati includono sia tipologie prettamente descrittive (es. `txamg.cdom`, `txamg.fdnm`) sia tipologie enumerative per la validazione e il workflow (es. `txamg.migt` o `txamg.mapl`). Esempi di Domini essenziali sono legati ai formati fisici ASCII (`txamg.fast`), alle regole di mapping (`txamg.mapl`) o al tracciamento logico del file (`txamg.dfil`).

Questi elementi compongono l'ossatura fondamentale sulla quale si poggia l'integrazione per la generazione del codice di migrazione e l'analisi dell'importer Infor Exchange.

# Capitolo 4

# Architettura

## 4.1 Panoramica dell'Architettura di Sistema

L'architettura del Migration Tool è strutturata per essere modulare e scalabile. Al cuore del sistema vi è una separazione netta tra il modulo di ingestione (Importer), il motore di elaborazione metadati, e l'esecutore dinamico (Generatore di Codice). Questo approccio garantisce l'isolamento tra il layout dei dati pre-esistenti e la struttura del database Infor CloudSuite.

## 4.2 Progettazione del Software

### 4.2.1 Design dell'Importer e del Parser dei Dati

L'Importer agisce come interfaccia di frontiera. È progettato per interpretare il formato proprietario di Infor Exchange, analizzando file di esportazione testuali o strutturati. Estraendo i metadati dai record, l'Importer popola la gerarchia dei *Datafiles* (`txamg110`) e dei relativi *Fields* (`txamg111`). Questo design a caricamento passivo rende possibile supportare sia architetture multitenant cloud, sia installazioni legacy on-premise.

### 4.2.2 Motore di Generazione del Codice

Il nucleo computazionale è rappresentato dal motore di generazione. Basato su librerie compilate (DLL come `txamgdll0012` e `txamgdll0013`), il modulo interpreta il dizionario di mapping (definito in `txamg203` o *MappedFields*) e traduce i legami e le regole di trasformazione in script nativi Baan C ad elevata velocità. Questo meccanismo previene la scrittura manuale del codice, garantendo il rispetto rigoroso dei framework CloudSuite: il codice viene pri-

ma generato, poi compilato via `bic_compiler`, ed infine agganciato ai *Batch Runs*.

### 4.2.3 Modulo di Integrazione Ad-hoc e Callbacks

Mentre la maggioranza dei flussi gode di standardizzazione automatica, alcune regole di business complesse richiedono sviluppi specifici. A tal fine, l'architettura espone interfacce *callback*. In appositi script di modulo, gli sviluppatori possono iniettare porzioni di codice chiamate asincronamente dall'engine. Questi script personalizzati godono degli stessi vantaggi transazionali dell'utility principale.

## 4.3 Scelte Tecnologiche

### 4.3.1 Linguaggi e Framework

L'intero ecosistema è sviluppato sfruttando l'Infor Tools Framework, programmando in **Baan C (sia 3GL che 4GL)**. L'adozione del paradigma *DAL (Data Access Layer)* ha semplificato molto l'interazione sicura con tabelle e riferimenti aziendali.

### 4.3.2 Librerie di Supporto e Strumenti di Build

Tutta l'orchestrazione della configurazione, logging (`txamgdll0011`) e le utility stringhe si basano sulle librerie del sistema ERP (DLL native Baan e tool come il `ttdll string`). Le build non prevedono toolchain esterni complessi (come Maven o MSBuild), ma si appoggiano direttamente al toolset interno dell'ERP, il quale gestisce i legami VRC (Version Release Customer) multitenant per garantire l'incapsulamento del codice generato al corretto Tenant.

# Capitolo 5

# Implementazione

## 5.1 Sviluppo dell'Importer Infor Exchange

### 5.1.1 Gestione dei Formati e Parsing

L'importer è stato sviluppato come un modulo d'ingestione che si interpone tra i file sorgenti (prodotti dai vecchi server o estratti tramite tool legacy) e le tabelle di definizione del nuovo sistema. La libreria dedicata al metadata management (`txamgdll0011`) prende in carico l'interpretazione dei metadati (quali formati delimitati o logiche ASCII a lunghezza fissa) operando conversioni granulari su base parametrica secondo l'enumerato `txamg.fast`. Ogni riga recuperata viene controllata per garantire la conformità delle codifiche testo e i troncamenti o padding dei campi vuoti.

### 5.1.2 Ricostruzione delle Definizioni dei File da Migrare

L'obiettivo secondario dell'importer è auto-compilare il setup dello stesso *Migration Tool*. Utilizzando le definizioni originarie dei file (le tabelle legacy), il programma crea dinamicamente o aggiorna una mappatura interna dei "Data Files" attraverso sessioni back-end come `txamg1110m000`. Le descrizioni, numerazioni dei campi (`txamg.fdno`) ed i tipi di dominio correlati (`txamg.atyp`) vengono riversati nella tabella di supporto `txamg111` e sono quindi subito pronti e precompilati per le successive attività di data-mapping manuale verso le tabelle target CloudSuite.

## 5.2 Implementazione del Generatore di Codice

### 5.2.1 Template Engine e Creazione Dinamica

La generazione dinamica del codice è gestita mediante apposite librerie DLL (*Dynamic Link Library*) fornite nel sistema Infor, quali `txamgdll0012` (Script Generator - Load dll) e `txamgdll0013` (Script Generator - Process dll). Tali librerie sono in grado di analizzare la configurazione definita nelle tabelle di mapping (ad esempio `txamg203`) per generare script eseguibili on-the-fly, riducendo drasticamente il tempo manuale richiesto per configurare regole di trasformazione standard e garantendo che il codice finale aderisca alla business logic.

### 5.2.2 Supporto alle Funzioni di Trasformazione Ad-hoc

Oltre alle regole predefinite, l'interprete ammette la dichiarazione e l'aggancio di funzioni custom. Durante il processo, eventuali routine di validazione avanzata o pulizia del dato possono essere inoltrate ad altre interfacce di esecuzione (come `txamg3200` - Execute Batch Runs). L'ausilio del componente `txamgdll0011` risulta critico, occupandosi in maniera focalizzata del management dei metadati e del log strutturato di eventuali errori (Error Logging).

## 5.3 Interfacce Utente (Sessioni)

L'interazione con l'utente avviene attraverso un set completo di sessioni dedicate, denominate *m-sessions* o *s-sessions* a seconda della loro finalità operativa o di supporto. Si evidenziano, a titolo d'esempio:

- **Setup iniziale:** la sessione `txamg0100m000` permette la parametrizzazione globale dello strumento.
- **Definizione Formati:** le sessioni relative a `txamg1110` gestiscono il popolamento dei formati dati elaborati.

- **Mapping Dati:** la definizione di conversione logica e associazioni campo-campo tra orgine e destinazione avviene nelle interfacce di visualizzazione tabelle target (`txamg2102m000`) e mapping campi (`txamg2103m000`).

## 5.4 Integrazione Documentale

### 5.4.1 Esportazione delle Specifiche di Migrazione

Il completamento formale del modulo d'implementazione sfocia nella sua interconnessione con lo strumento aziendale adibito alla documentazione del progetto. A valle della configurazione ed eventuale esecuzione esplorativa del batch (le sessioni ad albero `txamg2100` e `txamg3200`), l'utilità estrae il risultato dei mapping salvato nel `txamg203` (Tabella Mappings), formattandolo mediante layout predefiniti. Questa esportazione documenta puntualmente in PDF o markup analitici ogni singolo dominio trasferito, rendicontando sia i campi copiati in rapporto d'uguaglianza (*one-to-one*) (in accordo con `txamg.mapl`) sia le casistiche speciali per cui è stato inoculato codice personalizzato, semplificando nettamente la successiva manutenzione logica dell'architettura.

# Capitolo 6

# Testing e Validazione

## 6.1 Strategia di Testing

Il testing di un software per la migrazione dei dati è un aspetto cruciale. Data la sensibilità delle logiche del *Data Dictionary* di Infor CloudSuite in ambienti Cloud Multitenant, ogni anomalia nel record di inserimento avrebbe risvolti critici nel sistema target.

### 6.1.1 Test Unitari sui Moduli di Parsing

I moduli di base (DLL quali `txamgdll0011`) per l'estrazione e il metadata management degli Infor Exchange sono stati collaudati su porzioni ristrette e controllate di file. Verificando ogni singola colonna (`txamg1111`), la validazione include il corretto casting di tipi enumerati, decimali a virgola mobile e codifiche caratteri ASCII, simulando una diversità di input.

### 6.1.2 Test di Integrazione del Flusso di Migrazione

L'integrazione di end-to-end è validata eseguendo flussi di test governati dalle logiche di batch (`txamg2100` e `txamg3200`). I *Runs Errors* offrono immediato riscontro di blocchi logici o semantici. Sono stati eseguiti scenari in cui la validazione è forzata per respingere record privi dei riferimenti chiave, verificando lo scarto della linea incriminata senza però interrompere il caricamento dell'intero *Run*.

## 6.2 Verifica dell'Integrità dei Dati

### 6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud)

Ogni file migrato gode di tracciamento puntuale (`txamg301`, `txamg302`). Il matching prevede comparazioni del numero dei record letti, processati ed eventualmente rigettati rispetto alla fonte. I report prodotti (tramite moduli ad hoc come `txamgdll0011` per il log d'errore strutturato) forniscono la garanzia numerica dell'operazione.

## 6.3 Metriche e Miglioramenti Prestazionali

Sono stati misurati i tempi di generazione codice on-the-fly e i successivi cicli d'inserimento dati (Insert tramite DAL contro Insert diretta su DataBase). Nel primo caso, l'adozione delle *DLL script generator* (`txamgdll0012`, `txamgdll0013`) riduce a pochi secondi la generazione di mapping anche per tabelle target di grandi dimensioni. Dal lato inserimento, gli algoritmi limitano le scritture singole impiegando procedure *array-insert* o commit bilanciati, garantendo buone performance persino su server CloudSuite dislocati remotamente.

# Capitolo 7

# Conclusioni

## 7.1 Obiettivi Raggiunti e Risultati Ottenuti

Nel percorso di questo elaborato di Tesi, si è analizzato, implementato e documentato un completo *Migration Tool* (con pacchetto aziendale `txamg`) nativo nell'ambiente Infor LN. I requisiti sfidanti, legati in larga parte alla natura multitenant dei nuovi ambienti CloudSuite, sono stati assecondati offrendo un approccio parametrico e automatizzato. Mediante lo sviluppo delle DLL di generazione del codice dal dizionario mappato (ad esempio `txamgdll0012` e `txamgdll0013`) e dell'architettura per la lettura nativa Infor Exchange, si è conseguito l'obiettivo aziendale in termini di affidabilità, performance logica e generazione del codice in tempo reale, estirpando la componente di trascrizione metodica manuale e suscettibile d'errori.

## 7.2 Sfide Incontrate

Le principali complessità progettuali hanno risieduto nel mapping rigido imposto dal DAL (Data Access Layer) e dall'interfacciarsi ad entità Infor chiuse, in cui il Baan C (3GL e 4GL) risulta l'unica tecnologia d'aggancio valida. In un ecosistema di questa portata, simulare il compilatore interno garantendo una perfetta validazione del codice per ogni variante di tenant è risultato critico dal punto di vista dell'analisi e debug dei prototipi iniziali della libreria generatrice di log. Analogamente, uniformare codifiche e formati disomogenei proveniente dagli strati Legacy ha richiesto robusti componenti di parsing e una logica granulare di *Runs* di batch (`txamg200`).

## 7.3 Sviluppi Futuri

A dispetto dello standard d'eccellenza raggiunto, si ritiene possibile evolvere ulteriormente il tool in diverse direzioni. In primo luogo, lo schema architetturale per le estensioni Custom potrebbe venire affiancato da procedure *Machine Learning* (addestrate sui mapping legacy) in grado di suggerire i mapping probabilistici e i moduli di conversione per grandi volumi disarticolati di tabelle non ancora definite (senza l'input umano in interfacce `txamg2103`). Secondo, l'apertura delle specifiche tramite esportazione a modelli UML o grafi digitali, se automatizzata nativamente dal framework, fornirebbe reportistica in real-time durante la fase di studio del cliente prima di ogni operazione di load, riducendo di fatto tempi e costi totali e supportando i consulenti AlteaIN e Infor verso transizioni Cloud più agili.

# Bibliografia


[1] Y. Guo et al. «Multi-tenant cloud computing architecture: A review». In: *Journal of Ambient Intelligence and Humanized Computing* (2020) (cit. alle pp. 1, 3).

[2] *Infor Cloudsuite Enterprise*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/cloudsuite-enterprise` (cit. alle pp. 1, 3).

[3] *Infor LN - Cloud ERP for Manufacturing*. Accessed: 2026-04-16. 2026. URL: `https://www.infor.com/products/ln` (cit. alle pp. 1, 3).

[4] B. Johansson e F. Sudzina. «Challenges in Data Migration to ERP Systems». In: *Enterprise Information Systems* (2008) (cit. alle pp. 3, 6).

[5] Johny Morris. *Practical Data Migration*. BCS, The Chartered Institute for IT, 2012 (cit. alle pp. 3, 6).